Part III — Beyond automation: Intelligent agents

CHAPTER 7

AGENT ARCHITECTURES IN SECURE ENVIRONMENTS

PAULO H LEOCADIO

Introduction

Recent discourse on agentic AI increasingly emphasizes mechanisms such as prompt chaining, shared context protocols, and multi-agent coordination layers. Although terminology and implementations vary, these discussions are often framed in terms of emerging market patterns (e.g., model context mediation protocols and related orchestration frameworks). They converge on a common challenge: enabling AI systems to exchange, consolidate, and act on information across distributed environments without compromising governance, security, or accountability. This chapter treats these mechanisms not as trends or products but as structural responses to that challenge, examining how agent architectures are composed, constrained, and deployed in secure operational settings.

Throughout this chapter, data is treated as governed infrastructure and externalized state rather than as an autonomous actor; agency arises only from systems that interpret, decide, and act under explicit constraints.

7.1 From diffusers to autonomous threat agents

Diffusion models, by themselves, are not agents. They do not pursue objectives, maintain state, or act upon an environment. Yet in secure operational settings, diffusion-based models increasingly function as **perceptual and generative components** inside larger agentic systems. They are sophisticated engines of representation. However, when these models are embedded in a system that maintains state (S) and has a set of admissible actions (U), they transition from static files to active

participants. To formalize this transition in our cannon, we define an **Autonomous Agent** (***A***) as a function of its model, its environment, and its regulatory boundaries:

$$A = (M, S, U, \gamma)$$

Here, γ represents the governance constraints established in Chapter 6. By defining the agent this way, we ensure that agency is never **unbounded**. As (Russell and Norvig 2022) suggest that the rationality of an agent is tied to its performance measure; in cybersecurity, that measure is the successful compression of response latency within the bounds of γ.

Figure 7.1 shows how generative components expand the proposal space by producing hypotheses and counterfactuals, but do not directly authorize execution. Agency emerges only after proposals are filtered through decision logic and policy constraints before actions are taken.

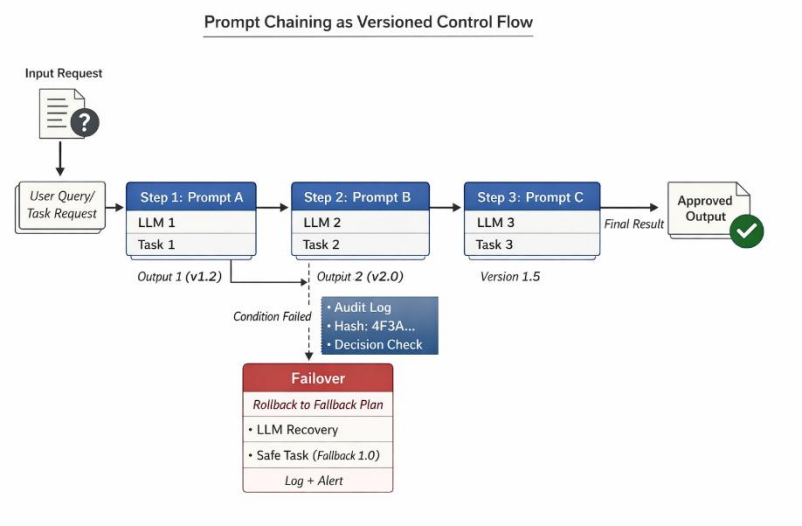

Figure 7.1 Agent loop with generative perception boundary.

The figure shows that, at the model level, diffusion architectures excel at structured generation: reconstructing signals, simulating counterfactuals, synthesizing plausible futures, and denoising incomplete observations. In security contexts, these capabilities are valuable precisely because they do not commit to action. Diffusers can generate threat hypotheses, simulate

attack progressions, or propose remediation paths without executing them. This makes them well-suited to serve as exploratory or analytical subsystems within a broader control loop.

Agency arises only when these generative outputs are embedded in a perception–decision–action pipeline governed by explicit policy. An autonomous threat agent is defined not by its use of a large or sophisticated model, but by its ability to (1) observe an environment, (2) interpret observations in context, (3) select actions aligned with objectives and constraints, and (4) execute or recommend those actions under bounded authority. Diffusion models contribute to steps (1) and (2), but they do not, on their own, satisfy the requirements of agency.

This distinction mirrors earlier transitions in distributed systems, where simulation and planning components informed execution engines without controlling them.

This distinction is critical in secure environments, where unbounded generative behavior is a liability rather than an asset. Autonomous agents in security operations centers, incident response systems, or critical infrastructure must be deterministic at the point of action, even when they rely on probabilistic reasoning upstream. Diffusion-based components, therefore, operate most safely in proposal space: generating candidate interpretations or response options that are then filtered, ranked, or vetoed by policy-driven decision modules.

As systems evolve toward autonomy, the architectural question shifts from "Which model is used?" to "Where is authority assigned?" In mature agent architectures, diffusion models are deliberately isolated from execution paths. They may inform decision-making but do not directly trigger containment actions, network reconfigurations, or enforcement measures. This separation preserves the benefits of generative flexibility while preventing stochastic outputs from causing irreversible operational consequences.

Another key trait of autonomous threat agents is persistence. Unlike stateless inference services, agents keep internal records of previous observations, decisions, and outcomes. Diffusion models can assist in updating these records by reconstructing degraded signals or predicting likely paths, but memory (not generation) grounds agency. Without explicit state management, a system might seem intelligent but remain fundamentally reactive.

Therefore, the transition from diffusers to autonomous agents does not occur solely through scaling models or chaining prompts. It occurs through the introduction of architectural boundaries: memory substrates that persist across interactions, policy layers that constrain actions, and control planes

that determine when automation yields to human authority. In secure environments, autonomy is not granted wholesale; it is incrementally assembled through constrained delegation.

Seen in this light, diffusion models are best understood as capability amplifiers rather than decision-makers. They expand the range of what an agent can consider without expanding the range of what it is permitted to do. This distinction underpins the design philosophy of secure agent architectures: maximize cognitive reach while minimizing uncontrolled execution.

Sources. *This section draws on prior work by* (Park, et al. 2023, Mialon, et al. 2023, Du, et al. 2023).

7.2 Prompt chaining, vector memory, and distributed policy enforcement

Modern agent architectures derive their apparent intelligence not from a single model invocation, but from the coordination of multiple subsystems (multi-agent coordination) that together impose structure, continuity, and constraints. Among these, prompt chaining, vector-based memory, and distributed policy enforcement form the core mechanisms through which agents reason, remember, and remain governable.

In practice, these mechanisms are rarely confined to a single process or host; prompt chains, memory access, and policy checks are increasingly distributed across networked services.

Multi-agent coordination is often criticized for its potential to create **escalation cascades**. To mitigate this, our architecture adopts the **Reflexion** framework (Shinn, et al. 2023). Rather than a single agent executing a command the moment a threshold is met, the primary responder must submit its plan to a **Critic** agent. This process of **verbal reinforcement** allows the system to self-correct. For example, if a responder agent proposes isolating a core database server, the Critic (acting on the governance metadata) will force a **reflection** cycle, requiring the responder to justify the blast radius before the action is committed to the control plane.

Prompt chaining is best understood as an explicit control structure rather than an internal reasoning process. In secure environments, prompts do not represent thoughts; they encode transitions between decision states. Each prompt invocation defines a bounded context, specifies an allowable operation, and produces an intermediate artifact that can be inspected, logged, or rejected. Chaining these prompts creates a deterministic execution graph layered atop probabilistic inference, enabling complex behavior to emerge without sacrificing control.

Figure 7.2 shows how prompt chaining externalizes agent reasoning into discrete, inspectable transitions. Each step produces auditable artifacts and can be constrained, rejected, or replayed under versioned governance.

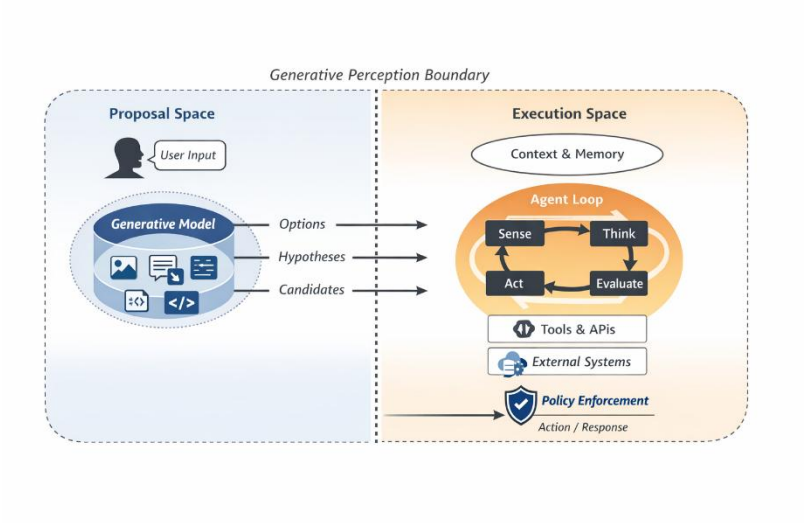

Figure 7.2 Prompt chain as versioned control flow.

This figure reframes prompt chaining as an engineering control structure. By making intermediate artifacts explicit, complex behavior becomes decomposable and governable, reducing the risk of hidden reasoning paths that cannot be audited or reproduced.

This framing is especially important in security-critical systems, where implicit or self-modifying reasoning paths are unacceptable. Prompt chains must be externally defined, auditable, and version-controlled, like policy logic. When treated as infrastructure rather than cognition, prompt chains decompose decisions into enforceable steps rather than serve as a mechanism for unconstrained deliberation.

Memory introduces distinct architectural concerns. Stateless inference systems respond only to their immediate inputs; agents, by contrast, must persist and retrieve contextual information across interactions. Vector-based memory systems offer a practical solution by externalizing state as queryable embeddings that can be selectively recalled. This approach avoids the brittleness of monolithic context windows while enabling agents to

ground decisions in historical patterns, prior incidents, or accumulated observations.

However, memory is also a primary attack surface. For this reason, memory systems in agent architectures resemble governed configuration stores rather than cognitive substrates.

In secure environments, vector stores must be treated as governed data assets rather than neutral caches. Memory insertion, retrieval, and expiration policies determine not only what an agent can recall but also what it is permitted to retain. Without constraints, vector memory can amplify bias, encode adversarial artifacts, or preserve outdated assumptions long after operational conditions have changed.

Figure 7.3 illustrates how vector memory externalizes state into a retrieval substrate, but it requires explicit governance of insertion, retrieval, and decay. Without controlled write/read paths, memory becomes a persistent attack surface and a vehicle for silent drift.

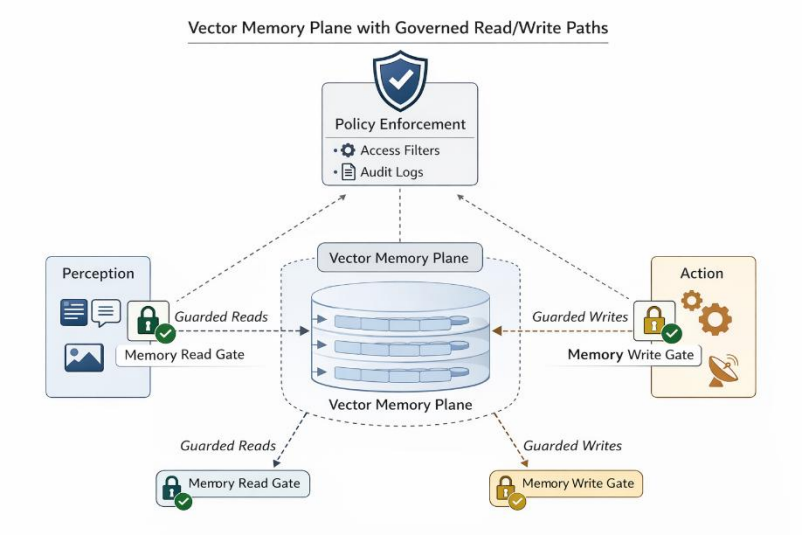

Figure 7.3 vector memory plane with governed write/read paths.

The figure emphasizes that memory governance is as important as model governance. Controlled insertion prevents pollution, filtered retrieval prevents irrelevant or adversarial recall, and decay mechanisms reduce latent influence from outdated operational contexts.

Effective agent architectures, therefore, impose strict boundaries on memory scope. Relevance thresholds and policy checks filter retrieval; insertion is limited to sanctioned events; and decay mechanisms ensure that obsolete context does not silently influence future decisions. In this sense, memory is not intelligence; it is a **delegated state,** subject to the same governance requirements as logs or configuration data.

Policy enforcement provides the final layer of stabilization. In distributed agent systems, no single component should have unilateral authority to act. Policies must be enforced across execution boundaries to ensure that decisions remain consistent as agents operate across services, environments, or cloud providers. This requires separating decision generation from decision authorization.

Figure 7.4 shows that in distributed agent systems, governance is enforced at shared control points rather than within individual agents. Agents generate action proposals, while authorization and constraint validation occur in a policy plane that remains independently versioned and updateable.

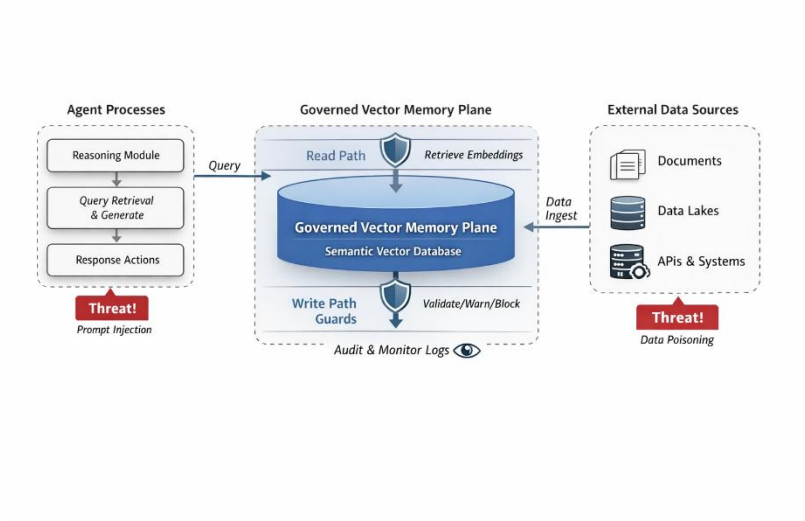

Figure 7.4 Distributed policy enforcement plane.

The figure makes clear that governance cannot be embedded solely in prompts or local logic. Centralized enforcement points prevent policy

divergence among distributed agents and enable rule updates without redeploying the entire agent fleet.

Distributed policy enforcement architectures typically externalize constraints to shared control planes or policy engines. Agents may propose actions, but enforcement points validate those proposals against current rules, risk thresholds, and authorization scopes. This design prevents local optimizations from violating global constraints and ensures policy updates propagate without requiring agent redeployment.

Failure modes arise when these layers are misaligned. Prompt chains can drift if not version-locked; memory can be corrupted if not curated; policies can desynchronize if enforcement is fragmented. In such cases, agent behavior may remain superficially coherent while progressively diverging from organizational intent. The system appears autonomous, yet accountability erodes.

The architectural lesson is clear: intelligence in agent systems emerges from composition rather than abstraction. Prompt chaining structures reasoning, vector memory provides continuity, and policy enforcement bounds actions. When these components are treated as first-class governance artifacts rather than auxiliary features, agents can operate autonomously without becoming unaccountable. When they are not, autonomy becomes indistinguishable from abdication.

Sources. *This section draws on prior work by* (Yao, et al. 2023, Du, et al. 2023, Shinn, et al. 2023, Łazuka, et al. 2022, Du, et al. 2023, Xu, Du and Li 2018).

7.3 Real-world design of minimal-latency decision agents

In secure operational environments, latency is not merely a performance metric; it is a control boundary. Decisions that arrive too late are functionally incorrect, regardless of their sophistication. As agents transition from advisory roles to active participants in security workflows, architectural emphasis shifts from model expressiveness to end-to-end response time, determinism, and locality.

Minimal-latency agent design starts by rejecting the "best model wins" assumption. In practice, the most capable model is often unusable if its inference path exceeds the decision budget imposed by the environment. Threat containment, access revocation, and automated remediation operate under strict temporal constraints, where milliseconds determine whether an action mitigates risk or merely documents it. Agent architectures must therefore be designed around decision deadlines rather than model benchmarks.

Figure 5 shows that latency constraints determine where agency can safely operate. Fast containment decisions require locality information and bounded action sets, whereas deeper deliberation shifts upstream to regional or central services when time budgets permit richer analysis.

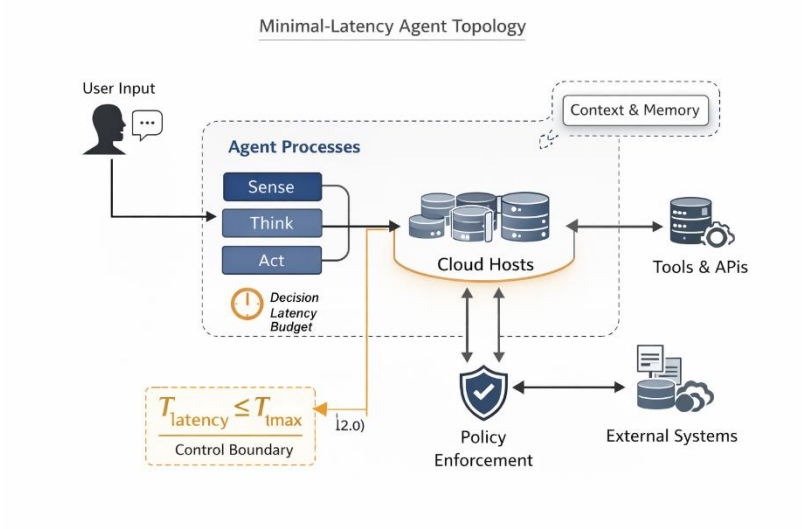

Figure 7.5 Minimal-latency agent topology

This figure illustrates why "best model" is often an inappropriate objective in secure environments. Architectural placement and predictability outweigh capability when decision deadlines are tight, and governance must remain enforceable even at execution speed.

This constraint has direct implications for deployment topology. Centralized inference services introduce unavoidable network latency, queuing delays, and coupling dependencies. In contrast, colocating lightweight decision agents near data sources (at the network edge, within regional clusters, or embedded in control planes) reduces response time and the failure surface. The architectural tradeoff favors proximity and predictability over raw model capacity.

Hardware heterogeneity further complicates latency-sensitive design. CPUs, GPUs, TPUs, and specialized accelerators each exhibit distinct performance characteristics under inference workloads. In many secure deployments, general-purpose CPUs (particularly modern ARM-based architectures) offer better tail latency and more predictable costs than shared

accelerators. Agents optimized for consistent response times often outperform more powerful but less deterministic alternatives.

Cross-cloud environments amplify these considerations. Secure organizations rarely operate within a single provider or region, and agent architectures must accommodate heterogeneous infrastructure without fragmenting behavior across providers or regions. This reality prioritizes portability and standardization over provider-specific optimizations. Agents must degrade gracefully by selecting locally available resources rather than assuming a homogeneous execution environment. Empirical evaluation of these tradeoffs often relies on vendor-published or practitioner benchmarks, underscoring the current gap between academic study and operational deployment realities.

Latency constraints also shape agents' internal structure. Reasoning depth must be bounded; prompt chains must be shallow; memory retrieval must be selective. In minimal-latency agents, complex deliberation is shifted upstream into planning or simulation phases, leaving execution-time agents to operate on precomputed policies and cached context. This separation preserves responsiveness while maintaining strategic adaptability elsewhere in the system.

Crucially, low latency does not justify reduced governance. In opposition, faster agents amplify the consequences of errors. Architectural safeguards (such as pre-approved action sets, confidence thresholds, and escalation gates) must be enforced at execution speed. Human override mechanisms must remain viable even when decisions are automated, ensuring that acceleration does not undermine accountability.

The practical outcome of these constraints is a class of agents that seem modest in isolation yet powerful in aggregate. Each agent performs a narrowly scoped role, operates within strict temporal and policy boundaries, and relies on upstream systems for context and planning. Autonomy emerges not from individual-agent complexity but from coordinated execution in a distributed, latency-aware architecture.

In secure environments, this design philosophy distinguishes automation that merely reacts from trustworthy agency. Minimal-latency decision agents succeed not by thinking harder but by acting just fast enough, exactly where and when it matters.

Sources. *This section draws on prior work by* (Evans, Stewart and Foster 2025, Sem 2025, Malladi, et al. 2024, Amazon Web Services 2024, Google Cloud 2023, Liang 2023).

7.4 Agent Design Examples:

The architectural patterns discussed in this chapter are best understood not as abstract ideals but as constraints embodied in concrete design choices. The following examples show how control, memory, and enforcement are realized in operational agent systems without prescribing specific tools, frameworks, or implementations. Each example isolates a single principle, demonstrating that autonomy is bounded by structure rather than diminished by it.

7.4.1 Versioned Prompt Chains as Explicit Control Flow

In secure agent architectures, prompt chains serve as externally defined control flow rather than implicit reasoning processes. Each step in the chain represents a discrete decision state, producing an artifact that can be inspected, logged, or replayed.

A simplified representation illustrates the principle:

```yaml
Step 1 (Chain v1.2):
  Input: Alert summary
  Output: Classified incident type

Step 2 (Chain v1.2):
  Input: Incident type + metadata
  Output: Proposed response category

Step 3 (Chain v1.2):
  Input: Response category
  Output: Action proposal
```

Each step is versioned and recoverable. If a downstream action is rejected or fails policy validation, the chain can be halted or resumed from a prior state without re-executing the entire sequence. This structure resembles distributed transaction patterns, in which partial progress is expected, and reversibility is a design requirement.

The architectural value lies in making transitions explicit. Reasoning becomes observable, and autonomy is expressed through controlled progression rather than through opaque inference.

7.4.2 Governed Vector Memory Write and Retrieval

Vector-based memory systems enable agents to retain contextual state across interactions, but persistence without governance quickly becomes a liability. In practice, memory systems are treated as regulated infrastructure rather than cognitive extensions.

A typical design separates memory access into sanctioned write and filtered read paths:

```pgsql
```

```
Write Path:
  Event → Validation → Embedding → Store (with TTL)

Read Path:
  Query → Similarity Search → Policy Filter → Context
Injection
```

Only approved events are eligible for storage, and each record includes metadata that specifies the scope, retention, and expiration. Retrieval is similarly constrained: relevance alone is insufficient without policy alignment. Records that exceed age thresholds, violate scope constraints, or originate from untrusted sources are excluded from consideration.

This design ensures that memory informs decisions without silently shaping them over time. The agent's apparent continuity is therefore governed, auditable, and resistant to the accumulation of adversarial or obsolete context.

7.4.3 Policy Enforcement at the Point of Execution

In distributed agent systems, decision generation and authorization are intentionally decoupled. Agents may propose actions, but execution authority resides elsewhere.

A minimal enforcement pattern can be expressed as:

```nginx
Proposed Action → Policy Check → {Allow | Deny |
Escalate}
```

Policy evaluation is independent of the agent's internal state or confidence. Even a correctly reasoned action is blocked if it exceeds authorization limits, violates risk thresholds, or conflicts with current operational constraints. Conversely, allowed actions are logged with sufficient context to support post hoc auditing.

This separation prevents local optimization from becoming a global failure. Autonomy is preserved at the proposal level, while accountability is enforced at execution time.

7.4.4 Latency-Bounded Agent Execution

Execution-time agents operate under strict temporal constraints. Decisions that exceed latency budgets are not deferred; they are aborted or escalated.

A simple guard condition captures this requirement:

$$T_{observe} + T_{retrieve} + T_{decide} + T_{enforce} \leq T_{max}$$

If the cumulative decision time exceeds the maximum allowable threshold, the agent yields control rather than acting on stale or incomplete context. More complex reasoning is shifted upstream into planning or simulation systems, leaving execution agents to operate within narrow, deterministic bounds.

This pattern reinforces a central theme of the chapter: in secure environments, speed is a form of safety, and restraint is a prerequisite for autonomy.

Synthesis

These examples show that effective agent design depends not on sophisticated internal cognition but on disciplined architectural boundaries. Control flow is externalized, memory is governed, execution is authorized, and latency is enforced. Together, these constraints enable agents to act autonomously while remaining accountable, a balance that defines operationally viable agentic systems.

Conclusion

Taken together, these architectural elements (Control, Memory, and Enforcement in Practice) redefine agentic AI as a systems problem rather than a modeling challenge. Autonomy is achieved not by increasing model complexity but by structuring perception, memory, policy, and execution across networked environments under explicit control. This framing provides a foundation for examining how such agents are deployed, governed, and integrated into cloud and platform ecosystems.

References

Amazon Web Services. 2024. “Graviton3 Benchmarks for AI Workloads." *Graviton3 Benchmarks for AI Workloads.*

Du, Y., L. Shuang, A. Torralba, J. B. Tenenbaum, and I. Mordatch. 2023. "Improving Factuality and Reasoning in Language Models through Multiagent Debate." *arXiv.* doi:https://doi.org/10.48550/arXiv.2305.14325.

Evans, K., E. Stewart, and G. Foster. 2025. "Comprehensive Review of Cloud-Driven Machine Learning: A Comparative Analysis of AWS, Azure, and Google Cloud." Tech. rep. https://www.techrxiv.org/users/875493/articles/1255263/master/file/data/p1/p1.pdf.

Google Cloud. 2023. "Cloud TPU v5p Performance and Comparison." Tech. rep. https://docs.cloud.google.com/tpu/docs/performance-guide.

Łazuka, M., T. Parnell, A. Anghel, and H. Pozidis. 2022. "Search-based Methods for Multi-Cloud Configuration." *Search-based Methods for Multi-Cloud Configuration.* arXiv. doi:10.48550/ARXIV.2204.09437.

Liang, J. 2023. "CrossCloud: Standardizing ML Deployment across Cloud Providers." *arXiv preprint.*

Malladi, S., T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora. 2024. "Fine-Tuning Language Models with Just Forward Passes." doi:https://doi.org/10.48550/arXiv.2305.17333.

Mialon, G., R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, Baptiste Rozière, et al. 2023. "Augmented Language Models: a Survey." *Transactions on Machine Learning Research* (arXiv) 2023. doi:10.48550/ARXIV.2302.07842.

Park, J. S., J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. 2023. "Generative Agents: Interactive Simulacra of Human Behavior." *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology.* ACM. 1-22. doi:10.1145/3586183.3606763.

Russell, S., and P. Norvig. 2022. *Artificial Intelligence: A Modern Approach.* Fourth. Pearson.

Sem, V. 2025. "Cloud AI Platforms: AWS vs. GCP vs. Azure for Machine-Learning Workloads." Tech. rep., Medium. https://medium.com/@virtualik/cloud-ai-platforms-aws-vs-gcp-vs-azure-for-machine-learning-workloads.

Shinn, N., F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. 2023. "Reflexion: language agents with verbal reinforcement learning."

Proceedings of the 37th International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc. doi:10.5555/3666122.3666499.

Xu, W., X. Du, and Z. Li. 2018. "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning." *2018 ACM Conference on Computer and Communications Security.* 1285-1298.

Yao, S., D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. 2023. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." *arXiv preprint arXiv:2305.10601* (arXiv). doi:10.48550/arXiv.2305.10601.